```
        const Doub rtol, bool dens);
    void step(const Doub htry,D &derivs);
    virtual void dy(VecDoub_I &y, const Doub htot, const Int k, VecDoub_O &yend,
        Int &ipt, D &derivs);
    void polyextr(const Int k, MatDoub_IO &table, VecDoub_IO &last);
    virtual void prepare_dense(const Doub h,VecDoub_I &dydxnew, VecDoub_I &ysav,
        VecDoub_I &scale, const Int k, Doub &error);
    virtual Doub dense_out(const Int i,const Doub x,const Doub h);
    virtual void dense_interp(const Int n, VecDoub_IO &y, const Int imit);
};
```

Detailed implementations of the member functions are given in a Webnote [6].

CITED REFERENCES AND FURTHER READING:

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §7.2.14.[1]

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.1.4 and §6.2.

Deuflhard, P. 1983, "Order and Stepsize Control in Extrapolation Methods," *Numerische Mathematik*, vol. 41, pp. 399–422.[2]

Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.[3]

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.[4]

Hairer, E., and Ostermann, A. 1990, "Dense Output for Extrapolation Methods," *Numerische Mathematik*, vol. 58, pp. 419–439.[5]

Numerical Recipes Software 2007, "StepperBS Implementations," *Numerical Recipes Webnote No. 21*, at `http://www.nr.com/webnotes?21` [6]

17.4 Second-Order Conservative Equations

Usually when you have a system of high-order differential equations to solve it is best to reformulate them as a system of first-order equations, as discussed in §17.0. There is a particular class of equations that occurs quite frequently in practice where you can gain about a factor of two in efficiency by differencing the equations directly. The equations are second-order systems where the derivative does not appear on the right-hand side:

$$y'' = f(x, y), \qquad y(x_0) = y_0, \qquad y'(x_0) = z_0 \tag{17.4.1}$$

As usual, y can denote a vector of values.

Stoermer's rule, dating back to 1907, has been a popular method for discretizing such systems. With $h = H/m$ we have

$$\begin{aligned} y_1 &= y_0 + h[z_0 + \tfrac{1}{2}hf(x_0, y_0)] \\ y_{k+1} - 2y_k + y_{k-1} &= h^2 f(x_0 + kh, y_k), \qquad k = 1, \ldots, m-1 \\ z_m &= (y_m - y_{m-1})/h + \tfrac{1}{2}hf(x_0 + H, y_m) \end{aligned} \tag{17.4.2}$$

Here z_m is $y'(x_0 + H)$. Henrici showed how to rewrite equations (17.4.2) to reduce roundoff error by using the quantities $\Delta_k \equiv y_{k+1} - y_k$. Start with

$$\begin{aligned} \Delta_0 &= h[z_0 + \tfrac{1}{2}hf(x_0, y_0)] \\ y_1 &= y_0 + \Delta_0 \end{aligned} \tag{17.4.3}$$

Then, for $k = 1, \ldots, m-1$, set

$$\begin{aligned} \Delta_k &= \Delta_{k-1} + h^2 f(x_0 + kh, y_k) \\ y_{k+1} &= y_k + \Delta_k \end{aligned} \tag{17.4.4}$$

Finally compute the derivative from

$$z_m = \Delta_{m-1}/h + \tfrac{1}{2} h f(x_0 + H, y_m) \tag{17.4.5}$$

Gragg again showed that the error series for equations (17.4.3) – (17.4.5) contains only even powers of h, and so the method is a logical candidate for extrapolation à la Bulirsch-Stoer.

Here is the `StepperStoerm` routine:

stepperstoerm.h

```
template <class D>
struct StepperStoerm : StepperBS<D> {
```

Stoermer's rule for integrating $y'' = f(x, y)$ for a system of equations.

```
    using StepperBS<D>::x; using StepperBS<D>::xold; using StepperBS<D>::y;
    using StepperBS<D>::dydx; using StepperBS<D>::dense; using StepperBS<D>::n;
    using StepperBS<D>::KMAXX; using StepperBS<D>::IMAXX; using StepperBS<D>::nseq;
    using StepperBS<D>::cost; using StepperBS<D>::mu; using StepperBS<D>::errfac;
    using StepperBS<D>::ysave; using StepperBS<D>::fsave;
    using StepperBS<D>::dens; using StepperBS<D>::neqn;
    MatDoub ysavep;
    StepperStoerm(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
        const Doub atol, const Doub rtol, bool dens);
    void dy(VecDoub_I &y, const Doub htot, const Int k, VecDoub_O &yend,
        Int &ipt,D &derivs);
    void prepare_dense(const Doub h,VecDoub_I &dydxnew, VecDoub_I &ysav,
        VecDoub_I &scale, const Int k, Doub &error);
    Doub dense_out(const Int i,const Doub x,const Doub h);
    void dense_interp(const Int n, VecDoub_IO &y, const Int imit);
};
```

Because the base class `StepperBS` is templated on the `derivs` class, the derived class `StepperStoerm` does not automatically inherit its member variables. This is the reason for the `using` declarations.

Note that in order to reuse the `StepperBS` code and make `StepperStoerm` a derived class, the arrays `y` and `dydx` are of length $2n$ for a system of n second-order equations. The values of y are stored in the first n elements of `y`, while the first derivatives are stored in the second n elements. The right-hand side f is stored in the first n elements of the array `dydx`, which therefore actually contains y''; the second n elements are unused.

The constructor has to redefine the costs A_k because there are half the number of function evaluations per step compared with the midpoint method:

stepperstoerm.h

```
template <class D>
StepperStoerm<D>::StepperStoerm(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
    const Doub atoll,const Doub rtoll, bool dens)
    : StepperBS<D>(yy,dydxx,xx,atoll,rtoll,dens),ysavep(IMAXX,n/2) {
```

Constructor. On input, `y[0..n-1]` contains y in its first `n`/2 elements and y' in its second `n`/2 elements, all evaluated at `x`. The vector `dydx[0..n-1]` contains the right-hand side function f (also evaluated at `x`) in its first `n`/2 elements. Its second `n`/2 elements are not referenced. Also input are the absolute and relative tolerances, `atol` and `rtol`, and the boolean `dense`, which is `true` if dense output is required.

```
    neqn=n/2;
    cost[0]=nseq[0]/2+1;
    for (Int k=0;k<KMAXX;k++)
        cost[k+1]=cost[k]+nseq[k+1]/2;
    for (Int i=0; i<2*IMAXX+1; i++) {
        Int ip7=i+7;
        Doub fac=1.5/ip7;
        errfac[i]=fac*fac*fac;
        Doub e = 0.5*sqrt(Doub(i+1)/ip7);
```

Number of equations.

Redefine cost: half as many function evaluations as Bulirsch-Stoer.

Coefficients for interpolation error are different too.

```
        for (Int j=0; j<=i; j++) {
            errfac[i] *= e/(j+1);
        }
    }
}
```

Here is the routine dy that implements Stoermer's rule:

stepperstoerm.h

```
template <class D>
void StepperStoerm<D>::dy(VecDoub_I &y, const Doub htot, const Int k,
    VecDoub_O &yend, Int &ipt, D &derivs) {
```

Stoermer step. Inputs are y, H, and k. The output is returned as `yend[0..2n-1]`. The counter `ipt` keeps track of saving the right-hand sides in the correct locations for dense output.

```
    VecDoub ytemp(n);
    Int nstep=nseq[k];
    Doub h=htot/nstep;                          Stepsize this trip.
    Doub h2=2.0*h;
    for (Int i=0;i<neqn;i++) {                  First step.
        ytemp[i]=y[i];
        Int ni=neqn+i;
        ytemp[ni]=y[ni]+h*dydx[i];
    }
    Doub xnew=x;
    Int nstp2=nstep/2;
    for (Int nn=1;nn<=nstp2;nn++) {             General step.
        if (dense && nn == (nstp2+1)/2) {
            for (Int i=0;i<neqn;i++) {
                ysavep[k][i]=ytemp[neqn+i];
                ysave[k][i]=ytemp[i]+h*ytemp[neqn+i];
            }
        }
        for (Int i=0;i<neqn;i++)
            ytemp[i] += h2*ytemp[neqn+i];
        xnew += h2;
        derivs(xnew,ytemp,yend);                Store derivatives temporarily in yend.
        if (dense && abs(nn-(nstp2+1)/2) < k+1) {
            ipt++;
            for (Int i=0;i<neqn;i++)
                fsave[ipt][i]=yend[i];
        }
        if (nn != nstp2) {
            for (Int i=0;i<neqn;i++)
                ytemp[neqn+i] += h2*yend[i];
        }
    }
    for (Int i=0;i<neqn;i++) {                  Last step.
        Int ni=neqn+i;
        yend[ni]=ytemp[ni]+h*yend[i];
        yend[i]=ytemp[i];
    }
}
```

The dense output routines are in a Webnote [1].

CITED REFERENCES AND FURTHER READING:

Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.